
NATIONAL ECONOMICS UNIVERSITY (NEU)
COLLEGE OF TECHNOLOGY
FACULTY OF DATA SCIENCE & ARTIFICIAL INTELLIGENCE (FDA)

OFFICIAL SOLUTIONS & MARKING RUBRIC

Course: Fundamental Programming Concepts in Python (DSAI1003)

Progress Exam: Numbers, Strings, and Operators in Python

Note for Instructors and Teaching Assistants: This document provides complete answer keys, rationale for conceptual questions, exact output traces, step-by-step program logic, reference Python implementations, and an itemized marking rubric designed for fair and consistent grading.

Section A: Numbers, Strings & Operators Concepts (20 points)

2 points per question. Total = 20 points.

Q#	Key	Conceptual Explanation & Rule Reference	Marks
Q1	C	Python identifiers must begin with a letter or an underscore and contain only alphanumeric characters and underscores. <code>_unit_price</code> is valid. <code>2nd_total</code> starts with a digit, <code>tax-rate</code> contains a hyphen (minus operator), and <code>class</code> is a reserved keyword.	2 pts
Q2	B	The assignment operator <code>=</code> evaluates the expression on the right-hand side using the current value of <code>count</code> , and then updates the storage location named <code>count</code> with the new value.	2 pts
Q3	B	Floor division (<code>//</code>) discards the fractional part and returns the quotient rounded down to the nearest integer. $19//4 = 4$ of type <code>int</code> . True division (<code>/</code>) would yield 4.75 of type <code>float</code> .	2 pts
Q4	A	Exponentiation <code>**</code> has highest precedence, followed by multiplication <code>*</code> , and then addition <code>+</code> . Thus, $2^3 = 8$, then $3 \times 8 = 24$, and finally $2 + 24 = 26$.	2 pts
Q5	C	For any base-10 integer, <code>n % 10</code> yields the remainder when divided by 10, which corresponds exactly to the rightmost units digit.	2 pts
Q6	B	Built-in functions such as <code>abs()</code> , <code>round()</code> , <code>min()</code> , and <code>max()</code> are part of Python's standard built-in namespace and do not require importing. Functions like <code>sqrt()</code> require <code>import math</code> .	2 pts
Q7	B	Computers represent floating-point numbers internally using binary (base 2). Many decimal fractions (like 0.1 and 0.2) have repeating binary representations, leading to tiny roundoff errors ($0.1 + 0.2 \approx 0.30000000000000004$).	2 pts
Q8	B	Python uses 0-based indexing from the left and negative indexing from the right. In "PYTHON", index 0 is 'P', index 1 is 'Y', and index -1 is the final character 'N'.	2 pts
Q9	B	String repetition <code>"5" * 2</code> has higher precedence than concatenation <code>+</code> , yielding "55". Then <code>"Data" + "55"</code> concatenates them into "Data55".	2 pts
Q10	B	Strings in Python are immutable . Calling <code>word.upper()</code> returns a new uppercase string "PYTHON", but does not modify <code>word</code> in place. Since the return value was not assigned back, <code>word</code> remains "python".	2 pts

Section B: Output Tracing, Type Conversion & Error Diagnosis (25 points)

Q11. Exact Output Tracing with Arithmetic & Modulo (5 points)

Solution for Q11

```
q: | 4 | r: | 3;
Check: True
Power diff: 2
```

Scoring Breakdown:

- **Line 1 (2 points):** Exact separator " | " between `q:`, `4`, `r:`, and `3`, ending with semicolon and newline (`;\n`).
- **Line 2 (1.5 points):** `Check: True` (calculates $4 \times 5 + 3 = 23$, which equals `a`, producing boolean `True`).
- **Line 3 (1.5 points):** `Power diff: 2` ($5^2 - 23 = 25 - 23 = 2$).

Q12. String Operations and Indexing (5 points)

Solution for Q12

- Output of `print(f"n = {n}"): n = 8` (1 point)
- Output of `print(f"Parts: {first}-{mid}-{last}"): Parts: C-u-r` (1.5 points)
Explanation: first = title[0] = 'C'; n//2 = 8//2 = 4, so title[4] = 'u'; last = title[-1] = 'r'.
- Output of `print(f"Result: {combo * 2}"): Result: curcur` (1.5 points)
Explanation: combo = ("Cur").lower() = "cur"; repeated twice is "curcur".
- **Explanation (1 point):** String indices must be integers. True division (`n / 2`) produces a float (e.g. `4.0`), which raises a `TypeError: string indices must be integers`. Integer floor division (`n // 2`) returns an `int`.

Q13. Diagnose and Correct Common Errors (5 points)

Solution for Q13

Code / Situation	Problem / Error	Correction / Valid Code
<pre>radius = input("Radius: ") area = 3.14159 * radius ** 2</pre>	<code>input()</code> returns a string (<code>str</code>). Exponentiation <code>**</code> and multiplication with float raise a <code>TypeError</code> .	<pre>radius = float(input("Radius: ")) area = 3.14159 * radius ** 2</pre>
<pre>s = "Python" s[0] = "J"</pre>	Strings are immutable; item assignment raises <code>TypeError: 'str' object does not support item assignment</code> .	<pre>s = "J" + s[1:]</pre>
<pre>total = 150 msg = "Total is: " + total</pre>	Concatenation <code>+</code> requires both operands to be strings. Combining <code>str</code> and <code>int</code> raises <code>TypeError</code> .	<pre>msg = "Total is: " + str(total) or msg = f"Total is: {total}"</pre>

Scoring: 1.5 pts for Row 1, 1.5 pts for Row 2, 2.0 pts for Row 3.

Q14. Formula Evaluation with the math Module (5 points)

Solution for Q14

Sides: 3.0, 4.0
Hypotenuse: 5.00
Triangle Area: 6.0

Scoring Breakdown:

- **Line 1 (1.5 points):** Sides: 3.0, 4.0 with 1 decimal digit each.
- **Line 2 (2 points):** Hypotenuse: 5.00 ($\sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5.0$, formatted as 5.00).
- **Line 3 (1.5 points):** Triangle Area: 6.0 ($0.5 \times 3.0 \times 4.0 = 6.0$).

Q15. Replacing Magic Numbers with Named Constants (5 points)**Solution for Q15**

- **Magic numbers identified (2 points):** 0.355 (liters per can) and 2.0 (liters per bottle).
- **PEP 8 naming convention (1 point):** Named constants are written in uppercase letters with words separated by underscores (ALL_CAPS_WITH_UNDERSCORES).
- **Refactored code (2 points):**

```
CAN_VOLUME = 0.355
BOTTLE_VOLUME = 2.0
total_liters = cans * CAN_VOLUME + bottles * BOTTLE_VOLUME
```


Section C: Program Design Using the Input → Process → Output Model (25 points)

Model Solution for Section C

Task 1: Identify the IPO Components (6 points)

- **Input(s):** Total change due to customer in cents (`total_cents`, an integer entered via keyboard). (2 pts)
- **Process:** (2 pts)
 1. Compute full 1-dollar bills: `dollars = total_cents // 100`
 2. Compute remaining cents: `rem_after_dollars = total_cents % 100`
 3. Compute 25-cent quarters: `quarters = rem_after_dollars // 25`
 4. Compute leftover cents: `cents_left = rem_after_dollars % 25`
 5. Compute total monetary value: `total_currency = total_cents / 100`
- **Output:** Display the original cents, count of 1-dollar bills, count of 25-cent quarters, leftover cents, and formatted currency amount `$X.XX`. (2 pts)

Task 2: Program Logic in Ordered Sequential Steps (7 points)

Step 1: **Input:** Prompt user with "Enter change in cents: " and convert to integer: `total_cents = int(input(...))`.

Step 2: **Compute Dollars:** Use floor division `dollars = total_cents // 100`.

Step 3: **Compute Dollar Remainder:** Use modulo remainder `remainder = total_cents % 100`.

Step 4: **Compute Quarters:** Use floor division `quarters = remainder // 25`.

Step 5: **Compute Leftover Cents:** Use modulo `cents_left = remainder % 25`.

Step 6: **Output:** Print a clean report using f-strings:

- `f"Total Cents: {total_cents}"`
- `f"Dollars ($1): {dollars}"`
- `f"Quarters (25c): {quarters}"`
- `f"Remaining Cents: {cents_left}"`
- `f"Total Amount: ${total_cents / 100:.2f}"`

Task 3: Hand Trace for `total_cents = 387` (6 points)

- `dollars = 387 // 100 = 3` (1.5 pts)
- `remainder = 387 % 100 = 87` (1.5 pts)
- `quarters = 87 // 25 = 3` (1.5 pts)
- `cents_left = 87 % 25 = 12` (1.5 pts)
- Verification: $3 \times 100 + 3 \times 25 + 12 = 300 + 75 + 12 = 387$ cents (\$3.87).

Task 4: Explain Design Choices (6 points)

a) Why use `int()` rather than `float()`? (3 points)

Coins and cents are discrete counts of whole physical units. Using `float()` could introduce binary floating-point roundoff inaccuracies (e.g., `387.0 % 100` might yield `86.99999999999999`), causing incorrect integer division counts.

b) Why use floor division `//` and remainder `%`? (3 points)

Section D: Applied Programming in Python (30 points)

Problem 1: Cylindrical Storage Tank Volume and Coating Cost (15 points)

Reference Implementation for Problem 1

```
import math

# Named constant
COATING_COST_PER_SQM = 14.50

# Input
tank_id = input("Enter tank ID: ")
radius = float(input("Enter radius in meters: "))
height = float(input("Enter height in meters: "))

# Process
base_area = math.pi * radius ** 2
wall_area = 2 * math.pi * radius * height
surface_area = 2 * base_area + wall_area
volume = base_area * height
total_cost = surface_area * COATING_COST_PER_SQM

# Output
print(f"Tank ID: {tank_id}")
print(f"Dimensions: Radius = {radius:.2f} m, Height = {height:.2f} m")
print(f"Storage Volume: {volume:.3f} m^3")
print(f"Total Surface Area: {surface_area:.2f} m^2")
print(f"Total Coating Cost: ${total_cost:.2f}")
```

Marking Rubric (15 points):

- **Import and Constants (2 pts):** Correct import math and uppercase constant COATING_COST_PER_SQM = 14.50.
- **Inputs and Conversions (3 pts):** Read tank_id as string, and convert radius and height to float.
- **Calculations (5 pts):** Correct formula for volume ($V = \pi r^2 h$, 2 pts), total surface area ($A = 2\pi r h + 2\pi r^2$, 2 pts), and cost (1 pt).
- **Output & Formatting (5 pts):** Correct text structure, radius/height/surface area with :.2f, volume with :.3f, and cost prefixed with \$ and :.2f.

Problem 2: Customer Voucher and Masked Account Statement (15 points)

Reference Implementation for Problem 2

```
# Named constant
DIVIDER_LENGTH = 42

# Input
full_name = input("Enter customer full name: ")
account_code = input("Enter 8-character account code: ")
account_balance = float(input("Enter account balance: "))

# Process
name_upper = full_name.upper()
name_length = len(full_name)
first_initial = full_name[0].upper()
masked_code = account_code[0] + account_code[1] + ("*" * 4) + account_code[-2] +
    account_code[-1]
top_border = "=" * DIVIDER_LENGTH
mid_divider = "-" * DIVIDER_LENGTH

# Output
print(top_border)
print("OFFICIAL CUSTOMER ACCOUNT VOUCHER")
print(mid_divider)
print(f"Customer Name: {name_upper} ({name_length} chars)")
print(f"Initial & Masked Code: {first_initial} | {masked_code}")
print(f"Current Balance: ${account_balance:.2f}")
print(top_border)
```

Marking Rubric (15 points):

- **Constant and Input (3 pts):** Constant `DIVIDER_LENGTH = 42`; inputs for name, code, and balance converted to float.
- **String Methods & Functions (3 pts):** Correctly applies `.upper()` and `len(full_name)`.
- **String Indexing & Masking (4 pts):** First initial extraction (`full_name[0]`); slicing/indexing first 2 and last 2 characters of `account_code` combined with `"*" * 4`.
- **Repetition & Divider Lines (2 pts):** Generates borders with `"=" * 42` and `"-" * 42`.
- **Output Structure & Formatting (3 pts):** Exact match to voucher format with `account_balance:.2f`.